
TRACE

Full implementation plan prompt

AI-powered intelligent CCTV monitoring

A complete coding-agent prompt for a student team: camera input, detection, tracking, zone rules, incident evidence, APIs, dashboard, integration and demonstration.

PYTHON / OPENCV / YOLO / BYTETRACK
FLASK / POSTGRESQL / REACT / FFMPEG

Includes all 28 prompt sections, 15 implementation phases, team responsibilities and the hackathon demo plan.

Use the prompt on pages 3 onward as instructions for a coding assistant. Examples describe the requested system; they are not a finished implementation.

Contents

1. CORE SYSTEM	3
2. FEATURES REQUIRED FOR MVP	4
3. OBJECT DETECTION	5
4. OBJECT TRACKING	6
5. ZONE SYSTEM	6
6. EVENT VALIDATION	7
7. INCIDENT GENERATION	7
8. INCIDENT SNAPSHOTS	8
9. INCIDENT VIDEO CLIPS	8
10. DATABASE	8
11. FLASK BACKEND	9
12. REAL-TIME ALERT SYSTEM	10
13. ALERT FLOOD PROTECTION	10
14. FALSE ALERT FEEDBACK	10
15. FAILSAFE DESIGN	11
16. PRIVACY REQUIREMENTS	11
17. REACT DASHBOARD	11
18. INCIDENT REVIEW PAGE	12
19. CAMERA CONFIGURATION	12
20. COMPUTER VISION WORKER	13
21. PROJECT STRUCTURE	13
22. MULTI-CAMERA DESIGN	14
23. PERFORMANCE	14
24. IMPLEMENTATION PHASES	14
25. TEAM DIVISION	16
26. HACKATHON MVP VS FUTURE	16
27. DEMO PLAN	17
28. WHAT I NEED FROM YOU	18
IMPLEMENTATION NOTE	19

TRACE - FULL IMPLEMENTATION PLAN PROMPT

You are acting as a senior computer-vision engineer, ML engineer, backend engineer, system architect, and hackathon mentor.

We are building a project called TRACE, an AI-powered intelligent CCTV monitoring system.

The goal is to add a software intelligence layer to existing CCTV cameras rather than replacing camera infrastructure.

Traditional CCTV systems mainly record footage and depend on humans continuously watching multiple feeds.

TRACE should monitor existing CCTV streams in real time, detect suspicious or abnormal activity, generate incidents, store evidence, and alert a human security operator.

The system should be designed primarily as a human-in-the-loop security assistant. The AI does not take autonomous enforcement actions.

1. CORE SYSTEM

The basic architecture is:

CCTV Camera

- > RTSP Stream
- > Video Capture
- > Frame Processing
- > YOLO Object Detection
- > ByteTrack Object Tracking
- > Behavior / Anomaly Detection
- > Persistence + Confidence Validation
- > Incident Generation
- > Snapshot / Video Clip
- > PostgreSQL
- > Flask REST API
- > React Monitoring Dashboard
- > Human Security Operator

The initial technology stack should remain:

Computer Vision

- Python
- OpenCV
- YOLO
- ByteTrack

Backend

- Flask
- REST APIs

Database

- PostgreSQL

Frontend

- React

Video Processing

- FFmpeg

Camera Input

- CCTV RTSP streams

Do not introduce unnecessary technologies or microservices unless absolutely required.

The priority is: working MVP > complicated architecture.

2. FEATURES REQUIRED FOR MVP

TRACE should initially detect the following events.

A. Restricted Area Intrusion

The administrator defines a polygon or rectangular restricted zone in the camera image.

If a tracked person enters that region and remains there beyond a configurable threshold, generate an incident.

Example:

Person detected

- > Track ID assigned
- > Bounding box intersects restricted polygon
- > Person remains inside > 2 seconds
- > Create intrusion incident

B. Loitering Detection

Detect when a tracked person remains inside a defined area longer than a configured period.

Example:

```
Track ID 17 enters area
timestamp recorded
Track remains in area
current_time - entry_time > threshold
-> LOITERING ALERT
```

The threshold should be configurable. Example: 10 seconds for testing; 30-120 seconds for production.

C. Crowd Detection

Detect when the number of people within a predefined region exceeds a configurable threshold.

Example: people_in_zone >= 8 for more than 5 seconds. Then create a crowd incident.

D. Generic Suspicious / Anomaly Framework

Design the architecture so additional detection modules can easily be added later.

Future modules may include:

- fighting detection
- fall detection
- abandoned-object detection
- unusual movement
- running
- restricted direction movement
- vehicle intrusion
- smoke/fire detection

Do NOT attempt to implement all of these in the MVP. Build an extensible event-detector interface instead.

Example:

```
class EventDetector:
    def process_frame(
        self,
        frame,
        detections,
        tracks,
        timestamp
    ):
        pass
```

Then implementations could include:

```
IntrusionDetector
LoiteringDetector
CrowdDetector
FightDetector
FallDetector
```

3. OBJECT DETECTION

Use YOLO for object detection.

For the MVP we mainly need person, but design the detection pipeline so additional classes can be enabled later.

The detector should return:

```
{
  "class": "person",
  "confidence": 0.91,
  "bbox": [x1, y1, x2, y2]
}
```

Explain:

- which YOLO version you recommend
- why
- pretrained vs custom-trained model
- confidence threshold
- inference resolution

- expected GPU/CPU requirements

Prefer a pretrained model for the hackathon MVP.

4. OBJECT TRACKING

Use ByteTrack. Each detected person should receive a persistent tracking ID.

Example:

```
Frame 1: Person -> ID 4; Person -> ID 9
Frame 2: Person -> ID 4; Person -> ID 9
```

Tracking information should include:

```
{
  "track_id": 4,
  "bbox": [],
  "confidence": 0.91,
  "first_seen": "...",
  "last_seen": "...",
  "current_zone": "entrance"
}
```

Explain exactly how YOLO detections are passed into ByteTrack.

5. ZONE SYSTEM

Implement configurable camera zones.

Each camera should support:

- Restricted Zone
- Loitering Zone
- Crowd Zone
- Ignore Zone

Store each zone as a polygon.

Example:

```
{
  "camera_id": 1,
  "name": "Server Room Entrance",
  "type": "restricted",
  "points": [
    [120, 140],
    [520, 140],
    [550, 600],
    [100, 600]
  ]
}
```

Use a point-in-polygon or bounding-box intersection method to determine whether a tracked person occupies a zone. Explain the best method.

6. EVENT VALIDATION

Do NOT create an alert from a single frame. Implement a persistence mechanism.

Example:

```
Person enters restricted zone
Frame 1 -> detected
Frame 2 -> detected
Frame 3 -> detected
...
threshold exceeded
  -> verified event
```

This reduces false alerts.

Each event detector should maintain temporary state per:

```
camera_id
track_id
event_type
```

Explain how to efficiently manage this state.

7. INCIDENT GENERATION

When an anomaly is confirmed, create an incident.

Incident schema should contain approximately:

```
{
  "incident_id": "...",
  "camera_id": "...",
  "event_type": "restricted_intrusion",
  "severity": "medium",
  "confidence": 0.91,
  "timestamp": "...",
  "track_ids": [4],
  "snapshot_path": "...",
  "clip_path": "...",
  "status": "unreviewed"
}
```

Possible event types:

```
INTRUSION
LOITERING
CROWD
FALL
FIGHT
ABANDONED_OBJECT
```

Possible statuses:

```
UNREVIEWED
VERIFIED
FALSE_ALERT
RESOLVED
```

8. INCIDENT SNAPSHOTS

When an event occurs, save a JPEG frame containing:

- bounding boxes
- Track IDs
- zone boundary
- timestamp
- event type

Example filename:

```
incidents/  
  2026/  
    10/  
      04/  
        camera_02_intrusion_173201.jpg
```

9. INCIDENT VIDEO CLIPS

Use FFmpeg. Maintain a short rolling video buffer.

When an incident occurs, save something similar to 5 seconds before incident + 10 seconds after incident. Result: 15-second incident clip.

Explain the best implementation for this.

Consider:

- OpenCV frame ring buffer
- OR FFmpeg segment recording

Recommend the simplest reliable approach.

10. DATABASE

Use PostgreSQL.

Design schemas for:

- cameras
- zones
- incidents
- alert_events
- system_users

Potential schema:

cameras

id
name
location
rtsp_url
status
created_at

zones

id
camera_id
name
zone_type
polygon
configuration
created_at

incidents

id
camera_id
event_type
severity
confidence
snapshot_path
clip_path
status
created_at
reviewed_at

alert_events

id
incident_id
delivery_method
delivery_status
created_at

Provide actual PostgreSQL CREATE TABLE statements.

11. FLASK BACKEND

Build REST APIs. At minimum:

```
GET    /api/cameras
POST   /api/cameras
GET    /api/cameras/:id

GET    /api/incidents
GET    /api/incidents/:id
PATCH /api/incidents/:id

GET    /api/zones
POST   /api/zones
PATCH /api/zones/:id
DELETE /api/zones/:id
```

For each endpoint provide:

- request body
- response body
- validation
- error handling
- database operations

12. REAL-TIME ALERT SYSTEM

When an incident occurs:

Computer vision worker

- > Incident created
- > Stored in PostgreSQL
- > Alert API
- > Dashboard receives notification

Recommend whether the MVP should use polling, Server-Sent Events, or WebSockets.

Select whichever is easiest and reliable for a hackathon implementation.

13. ALERT FLOOD PROTECTION

Multiple similar events occurring close together should not generate dozens of notifications. Implement event grouping.

For example:

Camera 5
Crowd alert
Crowd alert
Crowd alert
Crowd alert

Within 30 seconds becomes:

ONE INCIDENT
Event: Crowd
Duration: 43 seconds

Explain an event cooldown / deduplication strategy.

14. FALSE ALERT FEEDBACK

Every incident in the dashboard should have:

- Verify
- False Alert
- Resolve

If a security operator clicks False Alert, save that decision.

Example:

```
incident_status = FALSE_ALERT
```

Do NOT automatically retrain models during the hackathon.

However, structure the system so these annotations can later be used for:

- threshold tuning
- dataset generation
- model retraining

15. FAILSAFE DESIGN

AI must not be the sole decision maker.

If an AI component fails:

- previously detected incidents must remain stored
- the dashboard should indicate the component is offline
- queued alerts should not disappear
- humans remain responsible for reviewing incidents

Design health checks for:

- Camera connection
- YOLO inference service
- Tracker
- Database
- Backend

16. PRIVACY REQUIREMENTS

The system should intentionally avoid identity recognition.

Do NOT implement:

- Face recognition
- Identity tracking
- Biometric profiling

The system should primarily track behavior and anonymous object IDs. Example: Person #42 instead of John Smith.

Incident video clips should support automatic deletion after a configurable retention period. Example: delete after 7 days unless marked KEEP.

Design with India's DPDP principles in mind.

17. REACT DASHBOARD

Design a security-operations dashboard.

Pages:

```
/dashboard  
/cameras  
/incidents  
/incidents/:id  
/settings
```

Dashboard should show:

- Live cameras
- Recent incidents
- Incident severity
- Time
- Camera
- Event type
- Snapshot
- Status

Example incident card:

```
CAMERA 04  
Restricted Area Intrusion  
17:32:21  
Confidence: 91%  
[VIEW INCIDENT]
```

18. INCIDENT REVIEW PAGE

When an operator clicks an incident, show:

- Event type
- Camera
- Timestamp
- Snapshot
- 15-second video clip
- Confidence
- Track ID
- Zone
- Incident status

Buttons:

```
VERIFY  
FALSE ALERT  
RESOLVE
```

19. CAMERA CONFIGURATION

Provide a page where users can:

- add RTSP camera
- name camera

- specify location
- enable/disable monitoring
- draw zones on video frame
- configure thresholds

For example:

Loitering threshold: 30 sec
Crowd threshold: 10 people
Persistence threshold: 2 sec

20. COMPUTER VISION WORKER

The computer-vision loop should roughly operate like:

```
while True:
    frame = camera.read()
    detections = yolo.detect(frame)
    tracks = tracker.update(detections)
    events = anomaly_engine.process(
        frame,
        tracks
    )
    for event in events:
        if validator.confirm(event):
            incident = create_incident(event)
            save_snapshot(incident)
            save_clip(incident)
            store_database(incident)
            notify_dashboard(incident)
```

Turn this into production-quality modular Python code. Do not put everything in one file.

21. PROJECT STRUCTURE

Recommend a repository layout such as:

```
trace/
  backend/
    app.py
    api/
    models/
    services/
    database/
  vision/
    detector.py
    tracker.py
    camera.py
    zones.py
    anomaly_engine.py
    incident_manager.py
    detectors/
      intrusion.py
      loitering.py
      crowd.py
  frontend/
    src/
```

```
pages/  
components/  
services/  
hooks/  
storage/  
  incidents/  
config/  
docker/  
README.md
```

Provide the full recommended folder structure.

22. MULTI-CAMERA DESIGN

The architecture should eventually support multiple CCTV streams. For the MVP: 1-4 cameras is sufficient.

Explain how to process multiple cameras without blocking.

Possible approaches:

- one thread per camera
- multiprocessing
- async workers

Recommend the simplest approach that will still demonstrate properly.

23. PERFORMANCE

Explain expected performance requirements.

Include:

- FPS
- model size
- GPU requirements
- CPU fallback
- frame skipping
- inference resolution

For example, consider whether running AI inference at 10 FPS instead of 30 FPS is sufficient for CCTV anomaly detection.

24. IMPLEMENTATION PHASES

Give us a strict build order.

PHASE 1 - Video Input

Goal: RTSP/video -> OpenCV frames.

Success condition: Video plays reliably.

PHASE 2 - YOLO

Goal: frame -> person detection.

Success: Bounding boxes appear around people.

PHASE 3 - ByteTrack

Goal: person -> persistent ID.

Success: People maintain IDs while moving.

PHASE 4 - Zones

Goal: Allow restricted polygons to be defined.

Success: System can determine whether Track #X is inside Zone Y.

PHASE 5 - Intrusion Detection

Success: Walking into a restricted zone generates an event.

PHASE 6 - Loitering

Success: Remaining inside a region longer than threshold generates an event.

PHASE 7 - Crowd Detection

Success: Too many people inside a zone generates an event.

PHASE 8 - Incident Storage

Connect PostgreSQL. Store incidents.

PHASE 9 - Evidence

Generate: snapshot + video clip.

PHASE 10 - Backend

Expose incidents through Flask.

PHASE 11 - Dashboard

React dashboard displays incidents.

PHASE 12 - Real-Time Alerts

New incident appears automatically.

PHASE 13 - Operator Feedback

Implement: Verify, False Alert, Resolve.

PHASE 14 - Privacy / Cleanup

Implement automatic clip deletion.

PHASE 15 - Demo Optimization

Prepare controlled scenarios: restricted intrusion, loitering, crowd.

25. TEAM DIVISION

We currently have three major roles:

AI/ML & Computer Vision

Responsible for:

- YOLO
- ByteTrack
- zones
- anomaly engine
- intrusion
- loitering
- crowd detection

Backend & System Integration

Responsible for:

- Flask
- PostgreSQL
- APIs
- incident creation
- FFmpeg
- alerts
- connecting CV pipeline to backend

Frontend & UI/UX

Responsible for:

- React
- dashboard
- incident cards
- incident review
- camera management
- zone configuration UI

Show which tasks can happen in parallel so the team doesn't wait for each other.

26. HACKATHON MVP VS FUTURE

Divide everything into:

MUST HAVE

Prefer:

- YOLO person detection
- ByteTrack

- one CCTV/video stream
- restricted-area intrusion
- loitering
- crowd detection
- incident generation
- snapshot
- short video clip
- PostgreSQL
- Flask API
- React dashboard
- false-alert button

NICE TO HAVE

Examples:

- multiple cameras
- live notifications
- dashboard statistics
- zone editor
- email alerts

FUTURE

Do not build these unless time remains:

- fight detection
- fall detection
- abandoned-object detection
- multi-camera re-identification
- mobile app
- enterprise access-control integration
- large-scale edge/cloud deployment

27. DEMO PLAN

Design a 2-3 minute hackathon demo. For example:

Demo 1

Normal camera footage.

Explain: "TRACE adds intelligence to existing CCTV infrastructure."

Demo 2

Person walks into restricted region.

Show:

YOLO bounding box
-> Track ID
-> Zone intersection
-> Persistence timer
-> Intrusion detected

Demo 3

Dashboard immediately receives incident.

Show: snapshot, timestamp, camera, incident type, video clip.

Demo 4

Operator selects: Verified or False Alert.

Demo 5

Briefly show loitering/crowd detection.

End with: "Existing cameras record what happened. TRACE helps security teams notice it while it is happening."

28. WHAT I NEED FROM YOU

Now generate a complete implementation blueprint. Do NOT respond with generic descriptions.

For every stage give:

1. What we are building
2. Why it is needed
3. Libraries required
4. Files to create
5. Important classes/functions
6. Data flowing into the component
7. Data produced by the component
8. Example code
9. How to test it
10. Success criteria before moving to the next stage

After that, provide:

- A. Final architecture diagram
- B. Complete folder structure
- C. PostgreSQL schema
- D. Flask API specification
- E. Computer vision pipeline
- F. React dashboard structure
- G. Exact build order
- H. Team task distribution
- I. Integration strategy

- J. Testing checklist
- K. Deployment strategy
- L. Hackathon demo checklist

Assume we are university students building this for a hackathon, so optimize for a system we can actually finish, integrate and demonstrate, not an enterprise architecture that takes months to build.

Start with Phase 1: video input and computer vision architecture, and do not jump ahead until the underlying pipeline is logically clear.

IMPLEMENTATION NOTE

Start by building YOLO -> ByteTrack -> zones -> rule-based events. Once that pipeline works, adding fight/fall/unusual-behaviour models later becomes much easier.